In [12]:

```
# ================================================================
# 🟦 NAS100 Bot — Basse Fréquence / Forte Asymétrie (M15)
# ================================================================
# Régime   : Initial Balance Breakout + Liquidity Sweep
# TF signal: M15  →  Ticks Parquet (exécution)
# Session  : 15h30 – 18h00 (Paris / GMT+2)
# SL       : 1.5×ATR M15, cap absolu 25 pts
# TP1      : max(15 pts, 1.0×ATR)
# TP2      : 2.0×ATR
# TP3      : Trailing EMA20 activé après TP2
# ================================================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
import time
import pickle
import multiprocessing
from concurrent.futures import ThreadPoolExecutor, as_completed
from datetime import timedelta
from pathlib import Path
from collections import OrderedDict
import copy
from tqdm.notebook import tqdm

try:
    import polars as pl
    print(f'✅ Polars {pl.__version__} disponible')
except ImportError:
    raise ImportError('pip install polars')

warnings.filterwarnings('ignore')
plt.style.use('dark_background')

# ── Chemins ──────────────────────────────────────────────────────
PATH_M15        = Path('data/NAS100._M15_1Year.csv')
PATH_M1         = Path('data/NAS100._M1_1Year.csv')
PATH_TICKS      = Path('data/NAS100._Ticks_1Year.csv')
PATH_TICKS_PARQ = Path('data/NAS100._Ticks_1Year.parquet')
CACHE_IND       = Path('data/nas100_m15_indicators.pkl')

# ── Contractuel ──────────────────────────────────────────────────
CONTRACT_MULT  = 10.0
LOT_BASE       = 0.10
LOT_MIN        = 0.01
LOT_MAX        = 0.20
PNL_PER_POINT  = LOT_BASE * CONTRACT_MULT   # 1.00 $/pt

# ── Friction ─────────────────────────────────────────────────────
SLIPPAGE_PTS   = 2.0        # recalibré (vs 0.05 ancien)
MAX_SPREAD     = 1.5

# ── Session stricte GMT+2 ────────────────────────────────────────
GMT_OFFSET          = -1
```

```
SESSION_IB_START_H  = 14
SESSION_IB_START_M  = 0
SESSION_IB_END_H    = 14
SESSION_IB_END_M    = 30
SESSION_OPEN_H      = 15   # signal autorisé à partir de 15h30
SESSION_OPEN_M      = 30
SESSION_END_H       = 18

# ── Risk Management ──────────────────────────────────────
SL_ATR_MULT    = 1.5
SL_CAP_PTS     = 25.0       # cap absolu (vs 60 pts ancien)
TP1_MIN_PTS    = 15.0       # plancher anti-frictionnel
TP1_ATR_MULT   = 1.0
TP2_ATR_MULT   = 2.0
TRAIL_ATR_MULT = 0.8
DAILY_MAX_LOSS = -150.0     # (vs -200 ancien)
MAX_TRADES_DAY = 3          # (vs 20 ancien)
MAX_CONSEC_SL  = 2
COOLDOWN_SECS  = 3600       # 1h (vs 180s mode optimale)

# ── Initial Balance & Sweep ──────────────────────────────
IB_BUFFER_PTS  = 2.0
SWEEP_LOOKBACK = 20
EMA_TRAIL_SPAN = 20
ATR_PERIOD     = 14

print('✅ Configuration NAS100 M15 LF OK')
print(f'   Session  : {SESSION_OPEN_H}h{SESSION_OPEN_M:02d} → {SESSION_END_H}h00 GMT+2')
print(f'   SL       : {SL_ATR_MULT}×ATR, cap {SL_CAP_PTS} pts')
print(f'   TP1      : max({TP1_MIN_PTS} pts, {TP1_ATR_MULT}×ATR)')
print(f'   TP2      : {TP2_ATR_MULT}×ATR')
print(f'   TP3      : Trailing EMA{EMA_TRAIL_SPAN} post-TP2')
print(f'   Slippage : {SLIPPAGE_PTS} pts | Max trades : {MAX_TRADES_DAY}/jour')
```

```
✅ Polars 1.40.1 disponible
✅ Configuration NAS100 M15 LF OK
   Session  : 15h30 → 18h00 GMT+2
   SL       : 1.5×ATR, cap 25.0 pts
   TP1      : max(15.0 pts, 1.0×ATR)
   TP2      : 2.0×ATR
   TP3      : Trailing EMA20 post-TP2
   Slippage : 2.0 pts | Max trades : 3/jour
```

In [13]:
```
# ================================================================
# CELLULE 2 — Chargement M15 & M1
# ================================================================
def load_csv(path: Path, label: str) -> pd.DataFrame:
    df = pd.read_csv(path, parse_dates=['Time'])
    df.rename(columns={'TickVolume': 'Volume'}, inplace=True)
    df['Time'] = df['Time'] + pd.Timedelta(hours=GMT_OFFSET)
    df.set_index('Time', inplace=True)
    df.sort_index(inplace=True)
    df[['Open','High','Low','Close','Volume']] = \
        df[['Open','High','Low','Close','Volume']].astype(float)
    print(f'✅ {label} : {len(df):,} bougies | {df.index[0]} → 
{df.index[-1]}')
    return df

df_m15 = load_csv(PATH_M15, 'NAS100 M15')
df_m1  = load_csv(PATH_M1,  'NAS100 M1')

print('\n🔍 Format tick CSV :')
with open(PATH_TICKS, 'r') as f:
    for i, line in enumerate(f):
        print(f'  Ligne {i}: {line.strip()}')
        if i >= 3: break
print(f'\n📊 Fichier ticks : {PATH_TICKS.stat().st_size/1e9:.2f} GB')
```

```
✅ NAS100 M15 : 23,598 bougies | 2025-05-09 00:00:00 → 2026-05-08
22:45:00
✅ NAS100 M1 : 353,899 bougies | 2025-05-09 00:00:00 → 2026-05-08
22:59:00

🔍 Format tick CSV :
  Ligne 0: Time,Bid,Ask,Last,Volume
  Ligne 1: 2025.05.09 01:00:08.632,20089.55,20093.50,0.00,0
  Ligne 2: 2025.05.09 01:00:09.578,20089.05,20093.00,0.00,0
  Ligne 3: 2025.05.09 01:00:10.165,20090.30,20094.00,0.00,0

📊 Fichier ticks : 2.46 GB
```

In [14]:
```
# ================================================================
# CELLULE 2b — Conversion CSV → Parquet (UNE SEULE FOIS)
# ================================================================
def convert_ticks_to_parquet():
    if PATH_TICKS_PARQ.exists():
        print(f'✅ Parquet existant
({PATH_TICKS_PARQ.stat().st_size/1e9:.2f} GB) — ignoré')
        return
    print(f'⏳ Conversion CSV → Parquet...')
    t0 = time.time()
    with open(PATH_TICKS, 'r') as f:
        raw_cols = [c.strip().strip('<>') for c in
f.readline().strip().split(',')]
    time_col = next(c for c in raw_cols if 'time' in c.lower())
    bid_col  = next(c for c in raw_cols if 'bid'  in c.lower())
    ask_col  = next(c for c in raw_cols if 'ask'  in c.lower())
    (
        pl.scan_csv(PATH_TICKS, try_parse_dates=False,
infer_schema_length=500)
        .select([time_col, bid_col, ask_col])
        .rename({time_col: 'RawTime', bid_col: 'Bid', ask_col: 'Ask'})
        .with_columns([
            pl.col('Bid').cast(pl.Float64),
            pl.col('Ask').cast(pl.Float64),
            pl.col('RawTime')
              .str.to_datetime(format='%Y.%m.%d %H:%M:%S%.3f',
strict=False)
              .alias('Time'),
        ])
        .drop('RawTime')
        .sink_parquet(PATH_TICKS_PARQ, compression='snappy')
    )
    print(f'✅ Parquet créé en {time.time()-t0:.0f}s')

convert_ticks_to_parquet()
_TICKS_LAZY = pl.scan_parquet(PATH_TICKS_PARQ)
print('✅ LazyFrame Parquet NAS100 prêt')
```

```
✅ Parquet existant (0.42 GB) — ignoré
✅ LazyFrame Parquet NAS100 prêt
```

In [15]:

```
# ================================================================
# CELLULE 3 — Indicateurs M15 (Initial Balance + Liquidity Sweep)
# ================================================================
def compute_atr(df: pd.DataFrame, window: int = ATR_PERIOD) -> float:
    tr = pd.concat([
        df['High'] - df['Low'],
        (df['High'] - df['Close'].shift()).abs(),
        (df['Low']  - df['Close'].shift()).abs(),
    ], axis=1).max(axis=1)
    val = tr.rolling(window).mean().iloc[-1]
    return round(val, 2) if not np.isnan(val) else 0.0

def compute_ema(series: pd.Series, span: int) -> pd.Series:
    return series.ewm(span=span, adjust=False).mean()

def compute_initial_balance(df_m15: pd.DataFrame, date) -> dict:
    """High/Low de la fenêtre 14h00–14h30 (30 premières minutes)."""
    day_start = pd.Timestamp(date).replace(hour=SESSION_IB_START_H,
minute=SESSION_IB_START_M, second=0)
    day_end   = pd.Timestamp(date).replace(hour=SESSION_IB_END_H,
minute=SESSION_IB_END_M,   second=0)
    ib = df_m15[(df_m15.index >= day_start) & (df_m15.index < day_end)]
    if ib.empty:
        return {'ib_high': np.nan, 'ib_low': np.nan, 'ib_valid': False}
    return {'ib_high': round(ib['High'].max(), 2),
            'ib_low' : round(ib['Low'].min(),  2),
            'ib_valid': True}

def detect_liquidity_sweep(df_m15: pd.DataFrame, ts: pd.Timestamp,
                           lookback: int = SWEEP_LOOKBACK) -> dict:
    """Sweep haussier : dip sous swing low + clôture au-dessus.
       Sweep baissier : pic au-dessus swing high + clôture en-
dessous."""
    pos = df_m15.index.searchsorted(ts)
    if pos < lookback + 2:
        return {'sweep_bull': False, 'sweep_bear': False,
                'swing_low': np.nan, 'swing_high': np.nan}
    window  = df_m15.iloc[pos - lookback: pos]
    current = df_m15.iloc[pos - 1]
    swing_low  = window['Low'].iloc[:-1].min()
    swing_high = window['High'].iloc[:-1].max()
    sweep_bull = (current['Low'] < swing_low
                  and current['Close'] > swing_low
                  and current['Close'] > current['Open'])
    sweep_bear = (current['High'] > swing_high
                  and current['Close'] < swing_high
                  and current['Close'] < current['Open'])
    return {'sweep_bull' : sweep_bull,
            'sweep_bear' : sweep_bear,
            'swing_low'  : round(swing_low,  2),
            'swing_high' : round(swing_high, 2)}

def detect_ib_breakout(df_m15: pd.DataFrame, ts: pd.Timestamp, ib: dict)
-> dict:
    """Clôture M15 au-dessus/en-dessous de l'IB ± buffer."""
    if not ib.get('ib_valid', False):
        return {'bo_bull': False, 'bo_bear': False}
```

```
    pos = df_m15.index.searchsorted(ts)
    if pos < 1:
        return {'bo_bull': False, 'bo_bear': False}
    bar   = df_m15.iloc[pos - 1]
    ib_hi = ib['ib_high'] + IB_BUFFER_PTS
    ib_lo = ib['ib_low']  - IB_BUFFER_PTS
    return {'bo_bull': (bar['Close'] > ib_hi and bar['Close'] >
bar['Open']),
            'bo_bear': (bar['Close'] < ib_lo and bar['Close'] <
bar['Open'])}

def compute_indicators_m15(df_m15: pd.DataFrame, ts: pd.Timestamp,
                           ib: dict, window: int = 100) -> dict:
    pos = df_m15.index.searchsorted(ts)
    if pos < window:
        return {}
    sub   = df_m15.iloc[pos - window: pos]
    last  = sub.iloc[-1]
    price = last['Close']
    atr   = compute_atr(sub)
    if atr == 0:
        return {}
    ema20_s  = compute_ema(sub['Close'], 20)
    ema50_s  = compute_ema(sub['Close'], 50)
    ema200_s = compute_ema(sub['Close'], 200)
    vol_avg  = sub['Volume'].rolling(20).mean().iloc[-1]
    vol_ratio= round(last['Volume'] / vol_avg, 2) if vol_avg > 0 else
1.0
    sweep    = detect_liquidity_sweep(df_m15, ts)
    bo       = detect_ib_breakout(df_m15, ts, ib)
    momentum_4 = round(sub['Close'].iloc[-1] - sub['Close'].iloc[-5], 2)
\
                  if len(sub) >= 5 else 0.0
    return {
        'price'       : round(price, 2),
        'atr'         : atr,
        'ema20'       : round(ema20_s.iloc[-1], 2),
        'ema50'       : round(ema50_s.iloc[-1], 2),
        'ema200'      : round(ema200_s.iloc[-1], 2),
        'above_200'   : price > ema200_s.iloc[-1],
        'above_50'    : price > ema50_s.iloc[-1],
        'ema20_slope' : round(ema20_s.diff(3).iloc[-1], 2),
        'vol_ratio'   : vol_ratio,
        'momentum_4'  : momentum_4,
        'sweep_bull'  : sweep['sweep_bull'],
        'sweep_bear'  : sweep['sweep_bear'],
        'swing_low'   : sweep.get('swing_low',  price - atr * 2),
        'swing_high'  : sweep.get('swing_high', price + atr * 2),
        'bo_bull'     : bo['bo_bull'],
        'bo_bear'     : bo['bo_bear'],
        'ib_high'     : ib.get('ib_high', np.nan),
        'ib_low'      : ib.get('ib_low',  np.nan),
        'ib_valid'    : ib.get('ib_valid', False),
        'ema20_series': ema20_s.values,
    }

print('✅ Fonctions indicateurs M15 définies (IB + Sweep + EMA)')
```

```
✅ Fonctions indicateurs M15 définies (IB + Sweep + EMA)
```

In [16]:

```
# ================================================================
# CELLULE 4 — Direction, Score & Niveaux TP/SL (M15 LF)
# ================================================================
def determine_direction_m15(ind: dict) -> tuple:
    """Modèle A : Liquidity Sweep | Modèle B : IB Breakout."""
    if not ind:
        return None, 'no_data'
    # ── Modèle A : Sweep ──────────────────────────────────────
    if ind['sweep_bull']:
        if ind['above_200'] and ind['ema20_slope'] >= 0:
            return 'BUY', 'sweep_bull'
        elif ind['momentum_4'] > ind['atr'] * 0.3:
            return 'BUY', 'sweep_bull_momentum'
    if ind['sweep_bear']:
        if not ind['above_200'] and ind['ema20_slope'] <= 0:
            return 'SELL', 'sweep_bear'
        elif ind['momentum_4'] < -ind['atr'] * 0.3:
            return 'SELL', 'sweep_bear_momentum'
    # ── Modèle B : IB Breakout ────────────────────────────────
    if ind['ib_valid']:
        if ind['bo_bull'] and ind['above_200'] and ind['vol_ratio'] >=
1.2:
            return 'BUY',  'ib_breakout_bull'
        if ind['bo_bear'] and not ind['above_200'] and ind['vol_ratio']
>= 1.2:
            return 'SELL', 'ib_breakout_bear'
    return None, 'no_setup'

def compute_score_m15(ind: dict, direction: str) -> tuple:
    score = 0; reasons = []; is_buy = (direction == 'BUY')
    # Signal de base
    if is_buy and ind['sweep_bull']:         score += 30;
reasons.append('Sweep Haussier (+30)')
    elif not is_buy and ind['sweep_bear']:   score += 30;
reasons.append('Sweep Baissier (+30)')
    elif is_buy and ind['bo_bull']:          score += 22;
reasons.append('IB Breakout Haussier (+22)')
    elif not is_buy and ind['bo_bear']:      score += 22;
reasons.append('IB Breakout Baissier (+22)')
    # Tendance
    if is_buy and ind['above_200']:          score += 15;
reasons.append('Above EMA200 (+15)')
    elif not is_buy and not ind['above_200']:score += 15;
reasons.append('Below EMA200 (+15)')
    if is_buy and ind['above_50']:           score +=  8;
reasons.append('Above EMA50 (+8)')
    elif not is_buy and not ind['above_50']: score +=  8;
reasons.append('Below EMA50 (+8)')
    # Slope EMA20
    slope = ind['ema20_slope']
    if is_buy and slope > 0:                 score += 10;
reasons.append('EMA20 slope+ (+10)')
    elif not is_buy and slope < 0:           score += 10;
reasons.append('EMA20 slope- (+10)')
    # Volume
    vr = ind['vol_ratio']
    if vr >= 2.0:   score += 15; reasons.append(f'VolSpike x{vr:.1f}
```

```
(+15)')
    elif vr >= 1.4: score += 10; reasons.append(f'VolHigh x{vr:.1f}
(+10)')
    elif vr >= 1.0: score +=  5; reasons.append(f'VolOK x{vr:.1f} (+5)')
    # Momentum 1h
    mom = ind['momentum_4']; atr = ind['atr']
    if is_buy and mom > atr:           score += 12;
reasons.append(f'Momentum+ (+12)')
    elif not is_buy and mom < -atr:    score += 12;
reasons.append(f'Momentum- (+12)')
    return min(score, 100), reasons

def calculate_levels_m15(direction: str, entry: float, atr: float,
                         swing_high: float, swing_low: float) -> dict:
    """SL = min(swing ± buffer, 1.5×ATR), cap 25 pts.
       TP1 = max(15, 1.0×ATR) | TP2 = 2.0×ATR | TP3 = Trailing."""
    raw_risk = atr * SL_ATR_MULT
    if direction == 'BUY':
        sl   = round(min(swing_low - IB_BUFFER_PTS, entry - raw_risk),
2)
        sl   = round(max(sl, entry - SL_CAP_PTS), 2)
        risk = entry - sl
        tp1d = max(TP1_MIN_PTS, TP1_ATR_MULT * atr)
        return {'sl': sl, 'tp1': round(entry + tp1d, 2),
                'tp2': round(entry + TP2_ATR_MULT * atr, 2),
                'tp3': None, 'risk': round(risk, 2)}
    else:
        sl   = round(max(swing_high + IB_BUFFER_PTS, entry + raw_risk),
2)
        sl   = round(min(sl, entry + SL_CAP_PTS), 2)
        risk = sl - entry
        tp1d = max(TP1_MIN_PTS, TP1_ATR_MULT * atr)
        return {'sl': sl, 'tp1': round(entry - tp1d, 2),
                'tp2': round(entry - TP2_ATR_MULT * atr, 2),
                'tp3': None, 'risk': round(risk, 2)}

def compute_lot_dynamic(score: int, atr: float, median_atr: float) ->
float:
    mult = 1.0
    if score >= 80:   mult *= 1.2
    elif score <= 50: mult *= 0.8
    if median_atr > 0 and atr > 1.6 * median_atr:
        mult *= 0.7
    return max(LOT_MIN, min(LOT_MAX, round(LOT_BASE * mult, 2)))

print('✅ Fonctions signal, score, niveaux et sizing M15 LF définies')
```

```
✅ Fonctions signal, score, niveaux et sizing M15 LF définies
```

In [17]:

```
# ================================================================
# CELLULE 5 — Fetch Ticks + Simulateur Tick M15 (Trailing P3)
# ================================================================
def _fetch_ticks_numpy(signal_time_pc: pd.Timestamp, max_hours: int =
48) -> tuple:
    broker_start = signal_time_pc + timedelta(hours=-GMT_OFFSET)
    broker_end   = broker_start + timedelta(hours=max_hours)
    df = (_TICKS_LAZY
          .filter((pl.col('Time') >= broker_start) & (pl.col('Time') <=
broker_end))
          .select(['Bid','Ask'])
          .collect(streaming=True))
    if df.is_empty():
        return np.array([]), np.array([])
    return df['Bid'].to_numpy(), df['Ask'].to_numpy()

def simulate_trade_ticks(direction: str, entry_signal: float, levels:
dict,
                         bids: np.ndarray, asks: np.ndarray, atr:
float,
                         ema20_trail=None, slippage: float =
SLIPPAGE_PTS) -> list:
    if len(bids) == 0:
        return []
    spread_0 = asks[0] - bids[0]
    if spread_0 > MAX_SPREAD:
        return []
    entry  = round((asks[0] + slippage) if direction == 'BUY' else
(bids[0] - slippage), 2)
    delta  = entry - entry_signal
    sl_init= round(levels['sl']  + delta, 2)
    tp1    = round(levels['tp1'] + delta, 2)
    tp2    = round(levels['tp2'] + delta, 2)
    risk   = levels['risk']
    positions = [
        {'id':'P1','tp':tp1, 'sl':sl_init,'result':None,'exit_px':None,

'pnl':0.,'closed':False,'entry_real':entry,'slippage':round(abs(delta),4
),
         'mfe':0.,'mae':0.},
        {'id':'P2','tp':tp2, 'sl':sl_init,'result':None,'exit_px':None,

'pnl':0.,'closed':False,'entry_real':entry,'slippage':round(abs(delta),4
),
         'mfe':0.,'mae':0.},
        {'id':'P3','tp':None,'sl':sl_init,'result':None,'exit_px':None,

'pnl':0.,'closed':False,'entry_real':entry,'slippage':round(abs(delta),4
),
         'mfe':0.,'mae':0.,'trailing':False},
    ]
    tp1_hit = tp2_hit = False
    exec_px = bids if direction == 'BUY' else asks

    for k in range(len(exec_px)):
        if all(p['closed'] for p in positions):
            break
```

```
        px = exec_px[k]
        for p in positions:
            if not p['closed']:
                exc = (px - entry) if direction == 'BUY' else (entry -
px)
                p['mfe'] = max(p['mfe'], exc)
                p['mae'] = min(p['mae'], exc)
        p3 = positions[2]
        if not p3['closed'] and p3['trailing']:
            if ema20_trail is not None and len(ema20_trail) > 0:
                trail_level = ema20_trail[min(k, len(ema20_trail)-1)]
            else:
                trail_level = (px - atr * TRAIL_ATR_MULT if direction ==
'BUY'
                               else px + atr * TRAIL_ATR_MULT)
            new_sl = round(trail_level - atr*0.1 if direction=='BUY'
                           else trail_level + atr*0.1, 2)
            if direction == 'BUY'  and new_sl > p3['sl']: p3['sl'] =
new_sl
            if direction == 'SELL' and new_sl < p3['sl']: p3['sl'] =
new_sl
        for p in positions:
            if p['closed']: continue
            if p['id'] == 'P3' and p['trailing']:
                sl_h = (px <= p['sl']) if direction=='BUY' else (px >=
p['sl'])
                if sl_h:
                    p['closed']=True; p['exit_px']=p['sl']
                    rpts = (p['sl']-entry) if direction=='BUY' else
(entry-p['sl'])
                    p['result'] = 'TRAIL_CLOSE' if rpts > 0 else 'SL'
                    p['pnl']    = round(rpts * PNL_PER_POINT, 2)
                continue
           # --- Code Corrigé ---
            tp_v = p['tp']

            # 1. Vérification sécurisée du TP (ignore si None)
            if tp_v is not None:
                th = (px >= tp_v) if direction=='BUY' else (px <= tp_v)
            else:
                th = False # P3 n'a pas de limite fixe

            # 2. Vérification du SL (toujours présent)
            sh = (px <= p['sl']) if direction=='BUY' else (px >=
p['sl'])

            # 3. Résolution des conflits sur le même tick
            if th and sh and tp_v is not None:
                th = abs(tp_v-entry) < abs(p['sl']-entry)
                sh = not th
            if th:
                p['closed']=True; p['exit_px']=tp_v
                pts = (tp_v-entry) if direction=='BUY' else (entry-tp_v)
                p['result'] = p['id'].replace('P','TP')
                p['pnl']    = round(pts * PNL_PER_POINT, 2)
                if p['id']=='P1' and not tp1_hit:
                    tp1_hit = True
```

```
                    for o in positions:
                        if not o['closed']:
                            if direction=='BUY'  and entry > o['sl']:
o['sl'] = entry
                            elif direction=='SELL' and entry < o['sl']:
o['sl'] = entry
                if p['id']=='P2' and not tp2_hit and tp1_hit:
                    tp2_hit = True
                    if not positions[2]['closed']:
                        positions[2]['trailing'] = True
                        positions[2]['sl'] = (round(tp1+(tp2-
tp1)*0.25,2)
                                              if direction=='BUY'
                                              else round(tp1-(tp1-
tp2)*0.25,2))
            elif sh:
                p['closed']=True; p['exit_px']=p['sl']
                pts = abs(entry-p['sl'])
                is_be = abs(p['sl']-entry) < 0.5
                p['result'] = 'BE' if is_be else 'SL'
                p['pnl']    = 0. if is_be else round(-pts*PNL_PER_POINT,
2)
    last_px = exec_px[-1]
    for p in positions:
        if not p['closed']:
            pts = (last_px-entry) if direction=='BUY' else (entry-
last_px)
            p['result']='OPEN_END'; p['exit_px']=last_px
            p['pnl']=round(pts*PNL_PER_POINT,2); p['closed']=True
    return positions

print('✅ Fetch ticks + Simulateur M15 LF (trailing P3 EMA20) défini')
```

```
✅ Fetch ticks + Simulateur M15 LF (trailing P3 EMA20) défini
```

In [18]:
```
# ================================================================
# CELLULE 6 — Pré-calcul indicateurs M15 + cache IB par jour
# ================================================================
def build_ib_cache(df_m15: pd.DataFrame) -> dict:
    dates = df_m15.index.normalize().unique()
    cache = {d.date(): compute_initial_balance(df_m15, d.date()) for d
in dates}
    valid = sum(1 for v in cache.values() if v['ib_valid'])
    print(f'✅ IB calculée pour {valid}/{len(cache)} jours')
    return cache

def precompute_all_indicators(df_m15: pd.DataFrame,
                              ib_cache: dict, window: int = 100) ->
OrderedDict:
    if CACHE_IND.exists():
        print('📂 Cache M15 détecté — chargement...')
        with open(CACHE_IND, 'rb') as f:
            result = pickle.load(f)
        print(f'✅ {len(result):,} timestamps chargés')
        return result
    times   = df_m15.index.tolist()
    precomp = OrderedDict()
    t0 = time.time()
    with tqdm(total=len(df_m15)-window, desc='Pré-calcul M15',
unit='bougies', ncols=80) as pbar:
        for i in range(window, len(df_m15)):
            ts  = times[i]
            ib  = ib_cache.get(ts.date(), {'ib_valid': False})
            try:
                ind = compute_indicators_m15(df_m15, ts, ib, window)
                if ind: precomp[ts] = ind
            except Exception:
                pass
            pbar.update(1)
    print(f'\n✅ {len(precomp):,} timestamps en {time.time()-t0:.1f}s')
    with open(CACHE_IND, 'wb') as f:
        pickle.dump(precomp, f)
    return precomp

print('⏳ Calcul IB cache...')
ib_cache = build_ib_cache(df_m15)

if 'precomp_indicators' in globals() and precomp_indicators:
    print('✅ Indicateurs déjà en RAM — calcul ignoré.')
else:
    print('⏳ Pré-calcul indicateurs M15...')
    precomp_indicators = precompute_all_indicators(df_m15, ib_cache)
```

```
⏳ Calcul IB cache...
✅ IB calculée pour 258/259 jours
✅ Indicateurs déjà en RAM — calcul ignoré.
```

In [19]:
```
# ================================================================
# CELLULE 6b — Cache ticks (exec_time = ts + 15min, clôture M15)
# ================================================================
def _collect_signal_times(precomp: OrderedDict, min_score: int = 35) ->
list:
    times = []; last_ts = pd.Timestamp('1970-01-01')
    cur_date = None; trades_today = 0
    for ts, ind in precomp.items():
        day = ts.date()
        if day != cur_date:
            cur_date = day; trades_today = 0
        if trades_today >= MAX_TRADES_DAY: continue
        h, mn = ts.hour, ts.minute
        in_session = ((h > SESSION_OPEN_H or (h == SESSION_OPEN_H and mn
>= SESSION_OPEN_M))
                      and h < SESSION_END_H)
        if not in_session: continue
        if (ts - last_ts).total_seconds() < COOLDOWN_SECS: continue
        direction, _ = determine_direction_m15(ind)
        if direction is None: continue
        score, _ = compute_score_m15(ind, direction)
        if score < min_score: continue
        times.append(ts); last_ts = ts; trades_today += 1
    return times

def build_tick_cache(signal_times: list, max_hours: int = 48) -> dict:
    cache = {}
    print(f'📦 Chargement de {len(signal_times)} fenêtres de ticks...')
    t0 = time.time()
    for ts in tqdm(signal_times, desc='Cache ticks M15', unit='trade',
ncols=70):
        exec_time = ts + pd.Timedelta(minutes=15)
        bids, asks = _fetch_ticks_numpy(exec_time, max_hours)
        if len(bids) > 0:
            cache[ts] = (bids, asks)
    elapsed  = time.time() - t0
    total_mb = sum(b.nbytes + a.nbytes for b, a in cache.values()) / 1e6
    print(f'\n✅ Cache : {len(cache)} trades | {total_mb:.1f} MB |
{elapsed:.0f}s')
    return cache

print('🔍 Collecte timestamps candidats (score_min=35)...')
all_signal_times = _collect_signal_times(precomp_indicators,
min_score=35)
print(f'  → {len(all_signal_times)} signaux candidats\n')
print('⏳ Construction cache ticks...')
TICK_CACHE = build_tick_cache(all_signal_times)
```

```
🔍 Collecte timestamps candidats (score_min=35)...
  → 449 signaux candidats

⏳ Construction cache ticks...
📦 Chargement de 449 fenêtres de ticks...

Cache ticks M15:   0%|                    | 0/449 [00:00<?, ?
```

```
trade/s]


✅ Cache : 449 trades | 2209.3 MB | 13s
```

In [20]:

```
# ================================================================
# CELLULE 7 — ATR médiane 30j
# ================================================================
def compute_median_atr_30d(precomp: OrderedDict) -> pd.Series:
    records = [(ts, ind['atr']) for ts, ind in precomp.items()]
    s = pd.Series([r[1] for r in records],
                  index=pd.DatetimeIndex([r[0] for r in records]))
    return s.rolling('30D').median()

print('⏳ Calcul ATR médiane 30j...')
median_atr_series = compute_median_atr_30d(precomp_indicators)
print(f'✅ ATR médiane | min={median_atr_series.min():.1f} max=
{median_atr_series.max():.1f} pts')
```

```
⏳ Calcul ATR médiane 30j...
✅ ATR médiane | min=17.6 max=55.5 pts
```

In [21]:

```
# ================================================================
# CELLULE 8 — Moteur de backtest M15 LF (tick principal)
# ================================================================
def run_backtest_m15(precomp: OrderedDict, tick_cache: dict,
                     median_atr: pd.Series, score_min: int = 45,
                     label: str = 'NAS100 M15 LF') -> tuple:
    trades_log   = []
    daily_pnl    = 0.; trades_today = 0; consec_sl = 0
    last_sl_ts   = pd.Timestamp('1970-01-01')
    last_sig_ts  = pd.Timestamp('1970-01-01')
    cur_date     = None; daily_stopped = False
    fs = {k: 0 for k in ['no_session','no_setup','score_low',
                         'spread_rej','cooldown','consec_sl',
                         'daily_stop','freq_limit']}
    for ts, ind in tqdm(precomp.items(), desc=f'🚀 {label}',
unit='bougies', ncols=82):
        day = ts.date()
        if day != cur_date:
            cur_date=day; daily_pnl=0.; trades_today=0; consec_sl=0;
daily_stopped=False
        if daily_stopped:            fs['daily_stop'] += 1; continue
        if daily_pnl < DAILY_MAX_LOSS: daily_stopped=True;
fs['daily_stop'] += 1; continue
        if consec_sl >= MAX_CONSEC_SL: fs['consec_sl'] += 1; continue
        if trades_today >= MAX_TRADES_DAY: fs['freq_limit'] += 1;
continue
        h, mn = ts.hour, ts.minute
        in_session = ((h > SESSION_OPEN_H or (h == SESSION_OPEN_H and mn
>= SESSION_OPEN_M))
                      and h < SESSION_END_H)
        if not in_session: fs['no_session'] += 1; continue
        cooldown_eff = COOLDOWN_SECS * (1.5 ** consec_sl)
        if (ts - last_sig_ts).total_seconds() < cooldown_eff:
fs['cooldown'] += 1; continue
        direction, reject = determine_direction_m15(ind)
        if direction is None: fs['no_setup'] += 1; continue
        score, _ = compute_score_m15(ind, direction)
        if score < score_min: fs['score_low'] += 1; continue
        if ts not in tick_cache: continue
        entry_signal = ind['price']; atr = ind['atr']
        levels  = calculate_levels_m15(direction, entry_signal, atr,
                                       ind['swing_high'],
ind['swing_low'])
        med_atr = median_atr.get(ts, atr)
        lot     = compute_lot_dynamic(score, atr, med_atr)
        pnl_pt  = round(lot * CONTRACT_MULT, 4)
        bids, asks = tick_cache[ts]
        positions  = simulate_trade_ticks(
            direction, entry_signal, levels,
            bids=bids, asks=asks, atr=atr,
            ema20_trail=ind.get('ema20_series'),
            slippage=SLIPPAGE_PTS)
        if not positions: fs['spread_rej'] += 1; continue
        for p in positions:
            if p['pnl'] != 0.: p['pnl'] = round(p['pnl'] * (pnl_pt /
PNL_PER_POINT), 2)
        seq_pnl = sum(p['pnl'] for p in positions)
```

```
        daily_pnl = round(daily_pnl + seq_pnl, 2)
        trades_today += 1; last_sig_ts = ts
        if any(p['result'] == 'SL' for p in positions):
            consec_sl += 1; last_sl_ts = ts
        else:
            consec_sl = 0
        entry_real = positions[0].get('entry_real', entry_signal)
        slip_val   = positions[0].get('slippage', 0.)
        for p in positions:
            trades_log.append({

'open_time':ts,'day':day,'direction':direction,'signal':reject,

'entry_signal':entry_signal,'entry_real':entry_real,'slippage':slip_val,
                'atr':atr,'lot':lot,'pnl_per_pt':pnl_pt,'score':score,

'sl':levels['sl'],'tp1':levels['tp1'],'tp2':levels['tp2'],

'risk_pts':levels['risk'],'position':p['id'],'result':p['result'],
                'exit_px':p['exit_px'],'pnl':p['pnl'],
                'mfe':p.get('mfe',0.),'mae':p.get('mae',0.),
                'daily_pnl':daily_pnl,'hour':h,

'ib_high':ind.get('ib_high',np.nan),'ib_low':ind.get('ib_low',np.nan),
                'vol_ratio':ind['vol_ratio'],'consec_sl':consec_sl,
            })
    return pd.DataFrame(trades_log), fs

print('✅ Moteur de backtest NAS100 M15 LF défini')
```

```
✅ Moteur de backtest NAS100 M15 LF défini
```

In [22]:

```
# ================================================================
# CELLULE 9 — Lancement backtest
# ================================================================
print('🚀 Lancement backtest M15 LF...')
t0 = time.time()
df_trades, fs_tick = run_backtest_m15(
    precomp_indicators, TICK_CACHE, median_atr_series, score_min=45
)
n_seq = len(df_trades) // 3 if not df_trades.empty else 0
print(f'✅ Terminé en {time.time()-t0:.1f}s | {n_seq} séquences')
```

```
🚀 Lancement backtest M15 LF...

🚀 NAS100 M15 LF:   0%|                                        | 0/23498
[00:00<?, ?bougies/s]

✅ Terminé en 36.5s | 337 séquences
```

In [23]:

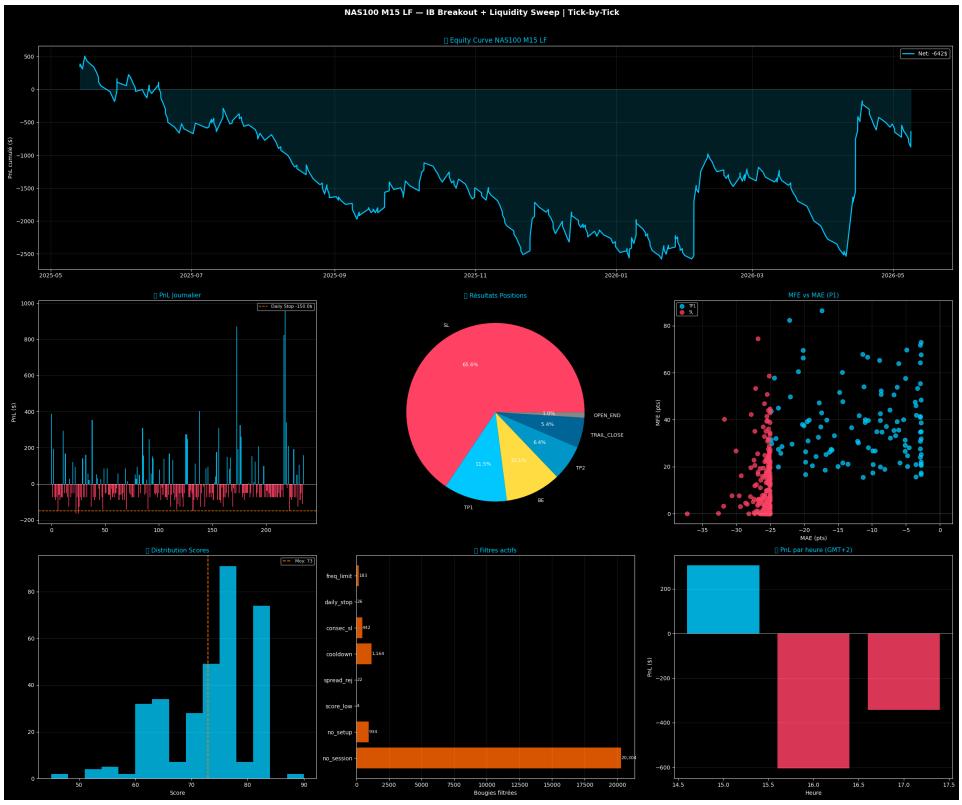
```
# ================================================================
# CELLULE 10 — Statistiques NAS100 M15 LF
# ================================================================
def compute_stats(df: pd.DataFrame, fs: dict, label: str) -> dict:
    if df.empty:
        print(f'⚠️ [{label}] Aucun trade.'); return {}
    seq   = df.groupby('open_time')['pnl'].sum()
    net   = round(df['pnl'].sum(), 2)
    gw    = df[df['pnl']>0]['pnl'].sum()
    gl    = abs(df[df['pnl']<0]['pnl'].sum())
    pf    = round(gw/gl, 2) if gl > 0 else float('inf')
    n_seq = len(seq)
    wr    = round((seq>0).sum()/n_seq*100, 1) if n_seq > 0 else 0
    equity= seq.cumsum()
    dd    = round((equity - equity.cummax()).min(), 2)
    n_days= max((df['open_time'].max()-df['open_time'].min()).days, 1)
    calmar= round(net*365/n_days/abs(dd), 2) if dd < 0 else float('inf')
    avg_win  = df[df['pnl']>0]['pnl'].mean() if (df['pnl']>0).any() else
0
    avg_loss = df[df['pnl']<0]['pnl'].mean() if (df['pnl']<0).any() else
0
    expectancy = round(wr/100*avg_win + (1-wr/100)*avg_loss, 2)
    print(f'\n{"="*62}')
    print(f'  [{label}]')
    print(f'  Période       : {df["open_time"].min().date()} →
{df["open_time"].max().date()}')
    print(f'  Séquences     : {n_seq}')
    print(f'  Net Profit    : {net:+.2f} $')
    print(f'  Profit Factor : {pf}')
    print(f'  Max Drawdown  : {dd:.2f} $')
    print(f'  Calmar Ratio  : {calmar}')
    print(f'  Winrate       : {wr} %')
    print(f'  Espérance/trd : {expectancy:+.2f} $')
    print(f'  Meilleure     : {seq.max():+.2f} $')
    print(f'  Pire          : {seq.min():+.2f} $')
    print(f'\n  Résultats par palier :')
    for tp in ['TP1','TP2','TRAIL_CLOSE','SL','BE','OPEN_END']:
        n  = (df['result']==tp).sum()
        pn = df[df['result']==tp]['pnl'].sum()
        if n > 0: print(f'    {tp:12s}: {n:4d}  | PnL : {pn:+.2f} $')
    if 'slippage' in df.columns:
        avg_slip = df.groupby('open_time')['slippage'].first().mean()
        print(f'\n  Friction       :')
        print(f'    Slippage moy : {avg_slip:.4f} pts')
        print(f'    Spread rej   : {fs.get("spread_rej",0)}')
    if 'signal' in df.columns:
        print(f'\n  Sources de signal :')
        for sig, cnt in df.groupby('open_time')
['signal'].first().value_counts().items():
            print(f'    {sig:<28}: {cnt}')
    print(f'{"="*62}')
    return {'net':net,'pf':pf,'dd':dd,'wr':wr,'calmar':calmar,
            'n_seq':n_seq,'equity':equity,'expectancy':expectancy}

stats_tick = compute_stats(df_trades, fs_tick, 'NAS100 M15 LF')
```

```
============================================================
  [NAS100 M15 LF]
  Période       : 2025-05-13 → 2026-05-08
  Séquences     : 337
  Net Profit    : -642.47 $
  Profit Factor : 0.96
  Max Drawdown  : -3085.97 $
  Calmar Ratio  : -0.21
  Winrate       : 34.4 %
  Espérance/trd : +5.28 $
  Meilleure     : +899.27 $
  Pire          : -90.00 $

  Résultats par palier :
    TP1         :  116  | PnL : +4001.12 $
    TP2         :   65  | PnL : +4632.71 $
    TRAIL_CLOSE :   55  | PnL : +2424.75 $
    SL          :  663  | PnL : -15097.50 $
    BE          :  102  | PnL : +0.00 $
    OPEN_END    :   10  | PnL : +3396.45 $

  Friction       :
    Slippage moy : 42.0075 pts
    Spread rej   : 22

  Sources de signal :
    ib_breakout_bull            : 179
    ib_breakout_bear            : 130
    sweep_bull_momentum         : 11
    sweep_bear_momentum         : 7
    sweep_bear                  : 5
    sweep_bull                  : 5
============================================================
```

In [24]:

```
# ================================================================
# CELLULE 11 — Graphiques analytiques
# ================================================================
if not df_trades.empty and stats_tick:
    fig = plt.figure(figsize=(24, 20))
    fig.suptitle('NAS100 M15 LF — IB Breakout + Liquidity Sweep | Tick-
by-Tick',
                 fontsize=14, fontweight='bold', color='white', y=0.99)
    eq = stats_tick['equity']
    ax1 = fig.add_subplot(3,3,(1,3))
    ax1.fill_between(eq.index, eq.values, alpha=0.15, color='#00ccff')
    ax1.plot(eq.index, eq.values, lw=2, color='#00ccff', label=f'Net:
{eq.iloc[-1]:+.0f}$')
    ax1.axhline(0, color='white', lw=0.5, linestyle='--', alpha=0.4)
    ax1.set_title('📈 Equity Curve NAS100 M15 LF', fontsize=12,
color='#00d4ff')
    ax1.set_ylabel('PnL cumulé ($)', color='white')
    ax1.tick_params(colors='white'); ax1.grid(alpha=0.15);
ax1.legend(fontsize=10)
    ax2 = fig.add_subplot(3,3,4)
    daily = df_trades.groupby('day')['pnl'].sum()
    bar_c = ['#00ccff' if v >= 0 else '#ff4466' for v in daily.values]
    ax2.bar(range(len(daily)), daily.values, color=bar_c, alpha=0.85)
    ax2.axhline(0, color='white', lw=0.5)
    ax2.axhline(DAILY_MAX_LOSS, color='#ff8800', lw=1.2, linestyle='--',
                label=f'Daily Stop {DAILY_MAX_LOSS}$')
    ax2.set_title('🗓️ PnL Journalier', fontsize=11, color='#00d4ff')
    ax2.set_ylabel('PnL ($)', color='white')
    ax2.tick_params(colors='white'); ax2.grid(alpha=0.15, axis='y');
ax2.legend(fontsize=8)
    ax3 = fig.add_subplot(3,3,5)
    rc  = df_trades['result'].value_counts()
    pal = {'TP1':'#00ccff','TP2':'#0099cc','TRAIL_CLOSE':'#006699',
           'SL':'#ff4466','BE':'#ffdd44','OPEN_END':'#888888'}
    ax3.pie(rc.values, labels=rc.index, autopct='%1.1f%%',
            colors=[pal.get(r,'#aaaaaa') for r in rc.index],
            textprops={'color':'white','fontsize':9})
    ax3.set_title('🎯 Résultats Positions', fontsize=11,
color='#00d4ff')
    ax4 = fig.add_subplot(3,3,6)
    if 'mfe' in df_trades.columns:
        df_p1 = df_trades[df_trades['position']=='P1'].copy()
        for res in df_p1['result'].unique():
            sub = df_p1[df_p1['result']==res]
            ax4.scatter(sub['mae'], sub['mfe'],
c=pal.get(res,'#aaaaaa'),
                        label=res, alpha=0.75, s=60)
    ax4.axhline(0, color='white', lw=0.5, alpha=0.3)
    ax4.axvline(0, color='white', lw=0.5, alpha=0.3)
    ax4.set_title('MFE vs MAE (P1)', fontsize=11, color='#00d4ff')
    ax4.set_xlabel('MAE (pts)', color='white'); ax4.set_ylabel('MFE
(pts)', color='white')
    ax4.tick_params(colors='white'); ax4.grid(alpha=0.15);
ax4.legend(fontsize=7)
    ax5 = fig.add_subplot(3,3,7)
    scores = df_trades.groupby('open_time')['score'].first()
    ax5.hist(scores.values, bins=15, color='#00ccff', alpha=0.8,
```

```
edgecolor='none')
    ax5.axvline(scores.mean(), color='#ff8800', lw=1.5, linestyle='--',
                label=f'Moy: {scores.mean():.0f}')
    ax5.set_title('📊 Distribution Scores', fontsize=11,
color='#00d4ff')
    ax5.set_xlabel('Score', color='white')
    ax5.tick_params(colors='white'); ax5.grid(alpha=0.15, axis='y');
ax5.legend(fontsize=8)
    ax6 = fig.add_subplot(3,3,8)
    lf = [k for k,v in fs_tick.items() if v > 0]
    vf = [fs_tick[k] for k in lf]
    bars6 = ax6.barh(lf, vf, color='#ff6600', alpha=0.85)
    ax6.set_title('🚦 Filtres actifs', fontsize=11, color='#00d4ff')
    ax6.set_xlabel('Bougies filtrées', color='white')
    ax6.tick_params(colors='white'); ax6.grid(alpha=0.15, axis='x')
    for bar, val in zip(bars6, vf):
        ax6.text(bar.get_width()+5, bar.get_y()+bar.get_height()/2,
                 f'{val:,}', va='center', color='white', fontsize=8)
    ax7 = fig.add_subplot(3,3,9)
    pnl_hour = df_trades.groupby('hour')['pnl'].sum()
    c7 = ['#00ccff' if v >= 0 else '#ff4466' for v in pnl_hour.values]
    ax7.bar(pnl_hour.index, pnl_hour.values, color=c7, alpha=0.85)
    ax7.axhline(0, color='white', lw=0.5)
    ax7.set_title('⏰ PnL par heure (GMT+2)', fontsize=11,
color='#00d4ff')
    ax7.set_xlabel('Heure', color='white'); ax7.set_ylabel('PnL ($)',
color='white')
    ax7.tick_params(colors='white'); ax7.grid(alpha=0.15, axis='y')
    plt.tight_layout(rect=[0,0,1,0.98])
    plt.savefig('nas100_m15_lf_results.png', dpi=150,
bbox_inches='tight',
                facecolor='#0d0d0d', edgecolor='none')
    plt.show()
    print('✅ Graphique sauvegardé → nas100_m15_lf_results.png')
```

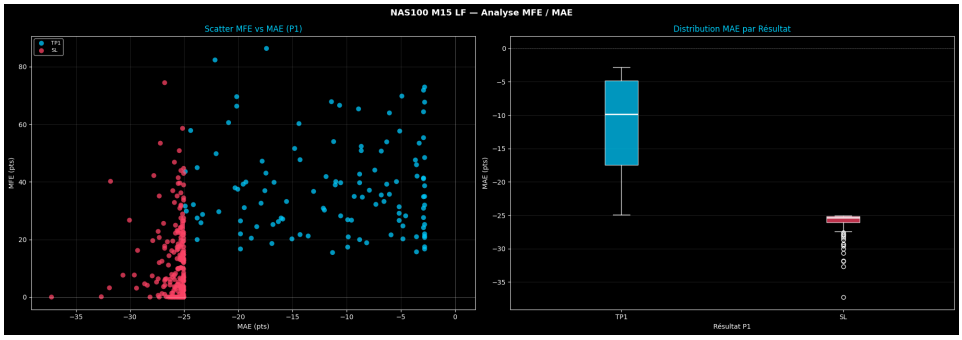


```
✅ Graphique sauvegardé → nas100_m15_lf_results.png
```

In [25]:
```
# ================================================================
# CELLULE 12 — MFE / MAE
# ================================================================
if not df_trades.empty and 'mfe' in df_trades.columns:
    df_p1 = df_trades[df_trades['position']=='P1'].copy()
    pal_res = {'TP1':'#00ccff','TP2':'#0099cc','TRAIL_CLOSE':'#006699',
               'SL':'#ff4466','BE':'#ffdd44','OPEN_END':'#888888'}
    fig2, axes2 = plt.subplots(1, 2, figsize=(20, 7))
    fig2.suptitle('NAS100 M15 LF — Analyse MFE / MAE',
                  fontsize=13, fontweight='bold', color='white')
    ax_s = axes2[0]
    for res in df_p1['result'].unique():
        sub = df_p1[df_p1['result']==res]
        ax_s.scatter(sub['mae'], sub['mfe'],
c=pal_res.get(res,'#aaaaaa'),
                     label=res, alpha=0.75, s=55, edgecolors='none')
    ax_s.axhline(0, color='white', lw=0.5, alpha=0.4)
    ax_s.axvline(0, color='white', lw=0.5, alpha=0.4)
    ax_s.set_xlabel('MAE (pts)', color='white'); ax_s.set_ylabel('MFE
(pts)', color='white')
    ax_s.set_title('Scatter MFE vs MAE (P1)', color='#00d4ff')
    ax_s.tick_params(colors='white'); ax_s.grid(alpha=0.15);
ax_s.legend(fontsize=8)
    ax_b = axes2[1]
    result_order = ['TP1','TP2','TRAIL_CLOSE','SL','BE']
    data_box   = [df_p1[df_p1['result']==r]['mae'].dropna().values
                  for r in result_order if (df_p1['result']==r).any()]
    labels_box = [r for r in result_order if (df_p1['result']==r).any()]
    bp = ax_b.boxplot(data_box, labels=labels_box, patch_artist=True,
                      medianprops=dict(color='white', lw=2))
    for patch, label in zip(bp['boxes'], labels_box):
        patch.set_facecolor(pal_res.get(label,'#aaaaaa'));
patch.set_alpha(0.75)
    ax_b.axhline(0, color='white', lw=0.5, linestyle='--', alpha=0.4)
    ax_b.set_xlabel('Résultat P1', color='white'); ax_b.set_ylabel('MAE
(pts)', color='white')
    ax_b.set_title('Distribution MAE par Résultat', color='#00d4ff')
    ax_b.tick_params(colors='white'); ax_b.grid(alpha=0.15, axis='y')
    print(f'\n📊 MFE/MAE (P1) :')
    print(f'  MFE moyen : {df_p1["mfe"].mean():+.1f} pts')
    print(f'  MAE moyen : {df_p1["mae"].mean():+.1f} pts')
    if (df_p1['result']=='SL').any():
        print(f'  MAE (SL)  : {df_p1[df_p1["result"]=="SL"]
["mae"].mean():+.1f} pts')
    if (df_p1['result']=='TP1').any():
        print(f'  MAE (TP1) : {df_p1[df_p1["result"]=="TP1"]
["mae"].mean():+.1f} pts')
    plt.tight_layout()
    plt.savefig('nas100_m15_mfe_mae.png', dpi=130, bbox_inches='tight',
facecolor='#0d0d0d')
    plt.show()
    print('✅ MFE/MAE sauvegardé → nas100_m15_mfe_mae.png')
```

```
📊 MFE/MAE (P1) :
  MFE moyen : +21.2 pts
```

```
MAE moyen : -20.8 pts
MAE (SL)  : -25.9 pts
MAE (TP1) : -11.3 pts
```



✅ MFE/MAE sauvegardé → nas100_m15_mfe_mae.png

In [26]:

```
# ================================================================
# CELLULE 13 — Sensibilité parallèle (Score Min × SL Mult)
# ================================================================
N_CORES = multiprocessing.cpu_count()
print(f'💻 CPU : {N_CORES} cœurs disponibles')

def _sensitivity_worker(args: tuple) -> dict:
    (precomp_items, tick_cache_local, sm, slm,
     pnl_pt_base, slip, sl_cap, tp1_min, tp1_atr, tp2_atr, ib_buf,
     max_trades, max_consec, cooldown_base, daily_max,
     ses_open_h, ses_open_m, ses_end_h, max_spread_val) = args
    import pandas as _pd; import numpy as _np
    from collections import OrderedDict as _OD
    precomp = _OD(precomp_items)
    def _levels(direction, entry, atr, sh, sl_v):
        raw = atr * slm
        if direction == 'BUY':
            sl = max(min(sl_v - ib_buf, entry - raw), entry - sl_cap)
            risk = entry - sl; tp1d = max(tp1_min, tp1_atr * atr)
            return
{'sl':sl,'tp1':entry+tp1d,'tp2':entry+tp2_atr*atr,'risk':risk}
        else:
            sl = min(max(sh + ib_buf, entry + raw), entry + sl_cap)
            risk = sl - entry; tp1d = max(tp1_min, tp1_atr * atr)
            return {'sl':sl,'tp1':entry-tp1d,'tp2':entry-
tp2_atr*atr,'risk':risk}
    def _sim(direction, entry_sig, lv, bids, asks):
        if asks[0]-bids[0] > max_spread_val: return []
        entry = round((asks[0]+slip) if direction=='BUY' else (bids[0]-
slip), 2)
        delta = entry - entry_sig
        sl=round(lv['sl']+delta,2); tp1=round(lv['tp1']+delta,2);
tp2=round(lv['tp2']+delta,2)
        pos =
[{'id':'P1','tp':tp1,'sl':sl,'pnl':0.,'result':None,'closed':False},

{'id':'P2','tp':tp2,'sl':sl,'pnl':0.,'result':None,'closed':False},

{'id':'P3','tp':None,'sl':sl,'pnl':0.,'result':None,'closed':False,'trai
ling':False}]
        tp1h=tp2h=False; ep = bids if direction=='BUY' else asks
        for px in ep:
            if all(p['closed'] for p in pos): break
            for p in pos:
                if p['closed']: continue
                if p['id']=='P3' and p['trailing']:
                    sl_h=(px<=p['sl']) if direction=='BUY' else
(px>=p['sl'])
                    if sl_h:
                        p['closed']=True; rpts=(p['sl']-entry) if
direction=='BUY' else (entry-p['sl'])
                        p['result']='TC' if rpts>0 else 'SL';
p['pnl']=round(rpts*pnl_pt_base,2)
                    continue
                th=(px>=p['tp']) if direction=='BUY' else (px<=p['tp'])
                sh=(px<=p['sl']) if direction=='BUY' else (px>=p['sl'])
                if th and sh: th=abs(p['tp']-entry)<abs(p['sl']-entry);
```

```
sh=not th
                if th:
                    p['closed']=True; pts=(p['tp']-entry) if
direction=='BUY' else (entry-p['tp'])
                    p['result']=p['id'].replace('P','TP');
p['pnl']=round(pts*pnl_pt_base,2)
                    if p['id']=='P1' and not tp1h:
                        tp1h=True
                        for o in pos:
                            if not o['closed']:
                                if direction=='BUY' and entry>o['sl']:
o['sl']=entry
                                elif direction=='SELL' and
entry<o['sl']: o['sl']=entry
                    if p['id']=='P2' and not tp2h and tp1h:
                        tp2h=True
                        if not pos[2]['closed']: pos[2]['trailing']=True
                elif sh:
                    p['closed']=True; pts=abs(entry-p['sl'])
                    p['result']='BE' if abs(p['sl']-entry)<0.5 else 'SL'
                    p['pnl']=0. if p['result']=='BE' else round(-
pts*pnl_pt_base,2)
        lp=ep[-1]
        for p in pos:
            if not p['closed']: p['result']='OE'; p['pnl']=round(((lp-
entry) if direction=='BUY' else (entry-lp))*pnl_pt_base,2);
p['closed']=True
        return pos
    def _dir(ind):
        if ind.get('sweep_bull') and ind.get('above_200'): return 'BUY'
        if ind.get('sweep_bear') and not ind.get('above_200'): return
'SELL'
        if ind.get('bo_bull') and ind.get('above_200') and
ind.get('vol_ratio',0)>=1.2: return 'BUY'
        if ind.get('bo_bear') and not ind.get('above_200') and
ind.get('vol_ratio',0)>=1.2: return 'SELL'
        if ind.get('sweep_bull') and
ind.get('momentum_4',0)>ind.get('atr',1)*0.3: return 'BUY'
        if ind.get('sweep_bear') and ind.get('momentum_4',0)<-
ind.get('atr',1)*0.3: return 'SELL'
        return None
    def _score(ind, direction):
        s=0; is_buy=direction=='BUY'
        if is_buy and ind.get('sweep_bull'): s+=30
        elif not is_buy and ind.get('sweep_bear'): s+=30
        elif is_buy and ind.get('bo_bull'): s+=22
        elif not is_buy and ind.get('bo_bear'): s+=22
        if is_buy and ind.get('above_200'): s+=15
        elif not is_buy and not ind.get('above_200'): s+=15
        if is_buy and ind.get('above_50'): s+=8
        elif not is_buy and not ind.get('above_50'): s+=8
        slope=ind.get('ema20_slope',0)
        if is_buy and slope>0: s+=10
        elif not is_buy and slope<0: s+=10
        vr=ind.get('vol_ratio',1.)
        if vr>=2.: s+=15
        elif vr>=1.4: s+=10
```

```
        elif vr>=1.: s+=5
        mom=ind.get('momentum_4',0); atr=ind.get('atr',1)
        if is_buy and mom>atr: s+=12
        elif not is_buy and mom<-atr: s+=12
        return min(s,100)
    pnls=[]; daily_pnl=0.; trades_today=0; consec_sl=0
    last_sig_ts=_pd.Timestamp('1970-01-01'); cur_date=None;
daily_stopped=False
    for ts, ind in precomp.items():
        day=ts.date()
        if day!=cur_date: cur_date=day; daily_pnl=0.; trades_today=0;
consec_sl=0; daily_stopped=False
        if daily_stopped or daily_pnl<daily_max: daily_stopped=True;
continue
        if consec_sl>=max_consec or trades_today>=max_trades: continue
        h,mn=ts.hour,ts.minute
        if not ((h>ses_open_h or (h==ses_open_h and mn>=ses_open_m)) and
h<ses_end_h): continue
        if (ts-last_sig_ts).total_seconds()<cooldown_base*
(1.5**consec_sl): continue
        direction=_dir(ind)
        if direction is None: continue
        score=_score(ind,direction)
        if score<sm: continue
        if ts not in tick_cache_local: continue
        lv=_levels(direction,ind['price'],ind['atr'],

ind.get('swing_high',ind['price']+25),ind.get('swing_low',ind['price']-2
5))
        bids,asks=tick_cache_local[ts]
        pos=_sim(direction,ind['price'],lv,bids,asks)
        if not pos: continue
        seq=sum(p['pnl'] for p in pos)
        daily_pnl=round(daily_pnl+seq,2); trades_today+=1;
last_sig_ts=ts
        if any(p['result']=='SL' for p in pos): consec_sl+=1
        else: consec_sl=0
        pnls.append(seq)
    if not pnls: return
{'score_min':sm,'sl_mult':slm,'net_profit':0,'n_seq':0,'winrate':0,'pf':
0,'calmar':0}
    arr=_np.array(pnls); net=round(arr.sum(),2); n_seq=len(arr)
    wr=round((arr>0).sum()/n_seq*100,1)
    gw=arr[arr>0].sum(); gl=abs(arr[arr<0].sum())
    pf=round(gw/gl,2) if gl>0 else 999.
    eq=_np.cumsum(arr); dd=round(float((eq-
_np.maximum.accumulate(eq)).min()),2)
    calmar=round(net/abs(dd),2) if dd<0 else 999.
    return
{'score_min':sm,'sl_mult':slm,'net_profit':net,'n_seq':n_seq,'winrate':w
r,'pf':pf,'calmar':calmar}

def run_sensitivity_parallel(precomp, tick_cache, score_range,
sl_range):
    precomp_list = list(precomp.items())
    combos = [(precomp_list, tick_cache, sm, slm,
               PNL_PER_POINT, SLIPPAGE_PTS, SL_CAP_PTS, TP1_MIN_PTS,
```

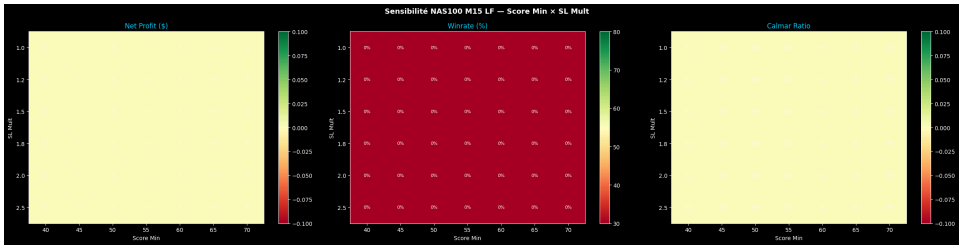
```
                TP1_ATR_MULT, TP2_ATR_MULT, IB_BUFFER_PTS,
                MAX_TRADES_DAY, MAX_CONSEC_SL, COOLDOWN_SECS,
DAILY_MAX_LOSS,
                SESSION_OPEN_H, SESSION_OPEN_M, SESSION_END_H,
MAX_SPREAD)
               for sm in score_range for slm in sl_range]
    results=[]; n_workers=min(N_CORES,len(combos))
    print(f'🚀 {len(combos)} combinaisons | {n_workers} workers CPU')
    t0=time.time()
    with ThreadPoolExecutor(max_workers=n_workers) as ex:
        futures={ex.submit(_sensitivity_worker,a):(a[2],a[3]) for a in
combos}
        with tqdm(total=len(combos), desc='Sensibilité M15',
unit='combo', ncols=70) as pbar:
            for fut in as_completed(futures):
                try: results.append(fut.result())
                except Exception as e:
                    sm,slm=futures[fut]

results.append({'score_min':sm,'sl_mult':slm,'net_profit':0,'n_seq':0,'w
inrate':0,'pf':0,'calmar':0})
                pbar.update(1)
    print(f'✅ Terminé en {time.time()-t0:.1f}s')
    return pd.DataFrame(results)

print('🔬 Analyse de sensibilité M15 LF PARALLÈLE...\n')
score_range = [40, 45, 50, 55, 60, 65, 70]
sl_range    = [1.0, 1.2, 1.5, 1.8, 2.0, 2.5]
df_sens = run_sensitivity_parallel(precomp_indicators, TICK_CACHE,
score_range, sl_range)
```

```
🖥️ CPU : 6 cœurs disponibles
🔬 Analyse de sensibilité M15 LF PARALLÈLE...

🚀 42 combinaisons | 6 workers CPU

Sensibilité M15:   0%|                          | 0/42 [00:00<?, ?
combo/s]

✅ Terminé en 0.9s
```

In [27]:

```
# ================================================================
# CELLULE 14 — Heatmaps de sensibilité
# ================================================================
pivot_net    = df_sens.pivot(index='sl_mult', columns='score_min',
values='net_profit')
pivot_wr     = df_sens.pivot(index='sl_mult', columns='score_min',
values='winrate')
pivot_calmar = df_sens.pivot(index='sl_mult', columns='score_min',
values='calmar')

fig3, axes3 = plt.subplots(1, 3, figsize=(24, 6))
fig3.suptitle('Sensibilité NAS100 M15 LF — Score Min × SL Mult',
              fontsize=13, fontweight='bold', color='white')

def _heatmap(ax, pivot, title, fmt, cmap='RdYlGn', vmin=None,
vmax=None):
    im = ax.imshow(pivot.values, aspect='auto', cmap=cmap, vmin=vmin,
vmax=vmax)
    ax.set_xticks(range(len(pivot.columns)));
ax.set_xticklabels(pivot.columns, color='white')
    ax.set_yticks(range(len(pivot.index)));
ax.set_yticklabels(pivot.index,   color='white')
    ax.set_xlabel('Score Min', color='white'); ax.set_ylabel('SL Mult',
color='white')
    ax.set_title(title, color='#00d4ff')
    for i in range(len(pivot.index)):
        for j in range(len(pivot.columns)):
            v = pivot.values[i,j]
            ax.text(j, i, fmt.format(v), ha='center', va='center',
fontsize=8,
                    color='black' if abs(v) <
abs(pivot.values).max()*0.6 else 'white')
    plt.colorbar(im, ax=ax)

_heatmap(axes3[0], pivot_net,    'Net Profit ($)', '{:.0f}')
_heatmap(axes3[1], pivot_wr,     'Winrate (%)',    '{:.0f}%', vmin=30,
vmax=80)
_heatmap(axes3[2], pivot_calmar, 'Calmar Ratio',   '{:.1f}')
plt.tight_layout()
plt.savefig('nas100_m15_sensitivity.png', dpi=130, bbox_inches='tight',
facecolor='#0d0d0d')
plt.show()

best_net    = df_sens.loc[df_sens['net_profit'].idxmax()]
best_calmar = df_sens.loc[df_sens['calmar'].idxmax()]
print(f'\n🏆 Meilleure combinaison (Net Profit) :')
print(f'   score_min={best_net["score_min"]:.0f} | sl_mult=
{best_net["sl_mult"]:.1f}'
      f' → Net: {best_net["net_profit"]:+.2f}$ | WR:
{best_net["winrate"]}% | Calmar: {best_net["calmar"]:.2f}')
print(f'\n🏆 Meilleure combinaison (Calmar) :')
print(f'   score_min={best_calmar["score_min"]:.0f} | sl_mult=
{best_calmar["sl_mult"]:.1f}'
      f' → Net: {best_calmar["net_profit"]:+.2f}$ | WR:
{best_calmar["winrate"]}% | Calmar: {best_calmar["calmar"]:.2f}')
```

```
print(f'\n🖥️ {N_CORES} cœurs utilisés | ✅ Sensibilité → nas100_m15_sensitivity.png')
```



🏆 Meilleure combinaison (Net Profit) :
score_min=50 | sl_mult=1.0 → Net: +0.00$ | WR: 0.0% | Calmar: 0.00

🏆 Meilleure combinaison (Calmar) :
score_min=50 | sl_mult=1.0 → Net: +0.00$ | WR: 0.0% | Calmar: 0.00

🖥️ 6 cœurs utilisés | ✅ Sensibilité → nas100_m15_sensitivity.png

In [28]:
```
# ================================================================
# CELLULE 15 — Notes de référence NAS100 M15 LF
# ================================================================
print("""
══════════════════════════════════════════════════════════════
📌 NOTES — NAS100 Bot M15 Basse Fréquence / Forte Asymétrie
══════════════════════════════════════════════════════════════

Architecture :
  M15 → signaux (IB Breakout + Liquidity Sweep)
  Ticks (Parquet LazyFrame) → exécution

Modèles d'entrée :
  A) Liquidity Sweep : spike sous swing low + clôture au-dessus
  B) IB Breakout : clôture M15 au-delà de l'IB ± 2 pts

Fenêtre temporelle (GMT+2) :
  14h00–14h30 : calcul de l'Initial Balance (non tradé)
  < 15h30     : interdit
  15h30–18h00 : fenêtre unique de trading
  ≥ 18h00     : hard block absolu

Friction simulée :
  BUY  : Ask + 2.0 pts | SELL : Bid - 2.0 pts
  Refus si spread > 1.5 pts
  exec_time = ts + 15 min (clôture bougie M15)

Gestion du risque :
  SL       : min(swing±buffer, 1.5×ATR), cap 25 pts
  TP1      : max(15 pts, 1.0×ATR)  → breakeven post-TP1
  TP2      : 2.0×ATR               → activation trailing P3
  TP3      : trailing EMA20 M15 (fallback ATR×0.8)
  Daily SL : -150 $
  Max/jour : 3 trades | Cooldown : 1h minimum
  Consec SL: 2 maximum (circuit breaker)

Sorties :
  nas100_m15_lf_results.png   — Equity + Journalier + Filtres
  nas100_m15_mfe_mae.png      — Scatter MFE/MAE
  nas100_m15_sensitivity.png  — Heatmaps Net/WR/Calmar

⚠️ Vérifier CONTRACT_MULT (VTMarkets) avant déploiement live.
══════════════════════════════════════════════════════════════
""")
```


```
══════════════════════════════════════════════════════════════
📌 NOTES — NAS100 Bot M15 Basse Fréquence / Forte Asymétrie
══════════════════════════════════════════════════════════════

Architecture :
  M15 → signaux (IB Breakout + Liquidity Sweep)
  Ticks (Parquet LazyFrame) → exécution

Modèles d'entrée :
  A) Liquidity Sweep : spike sous swing low + clôture au-dessus
  B) IB Breakout : clôture M15 au-delà de l'IB ± 2 pts
```

```
Fenêtre temporelle (GMT+2) :
  14h00–14h30 : calcul de l'Initial Balance (non tradé)
  < 15h30     : interdit
  15h30–18h00 : fenêtre unique de trading
  ≥ 18h00     : hard block absolu

Friction simulée :
  BUY  : Ask + 2.0 pts | SELL : Bid - 2.0 pts
  Refus si spread > 1.5 pts
  exec_time = ts + 15 min (clôture bougie M15)

Gestion du risque :
  SL       : min(swing±buffer, 1.5×ATR), cap 25 pts
  TP1      : max(15 pts, 1.0×ATR)  → breakeven post-TP1
  TP2      : 2.0×ATR               → activation trailing P3
  TP3      : trailing EMA20 M15 (fallback ATR×0.8)
  Daily SL : -150 $
  Max/jour : 3 trades | Cooldown : 1h minimum
  Consec SL: 2 maximum (circuit breaker)

Sorties :
  nas100_m15_lf_results.png   — Equity + Journalier + Filtres
  nas100_m15_mfe_mae.png      — Scatter MFE/MAE
  nas100_m15_sensitivity.png  — Heatmaps Net/WR/Calmar

⚠️ Vérifier CONTRACT_MULT (VTMarkets) avant déploiement live.
```